UNIT I – FUNDAMENTALS OF COMPUTER SYSTEMS AND ISAS

Introduction to Computer Architecture: Von Neumann & amp; Harvard Models, Evolution of ISAs: CISC, RISC,and the role of ARM, x86 – Components of a computer system , Instruction Set Architectures (ISA): Design Principles, Addressing Modes and Instruction Formats, Assembly Programming Basics (ARM/x86),Performance Metrics: CPI, MIPS, Amdahl's Law, Real-world ISA Case Study: ARM Cortex vs. Intel x86,simple assembly programs.

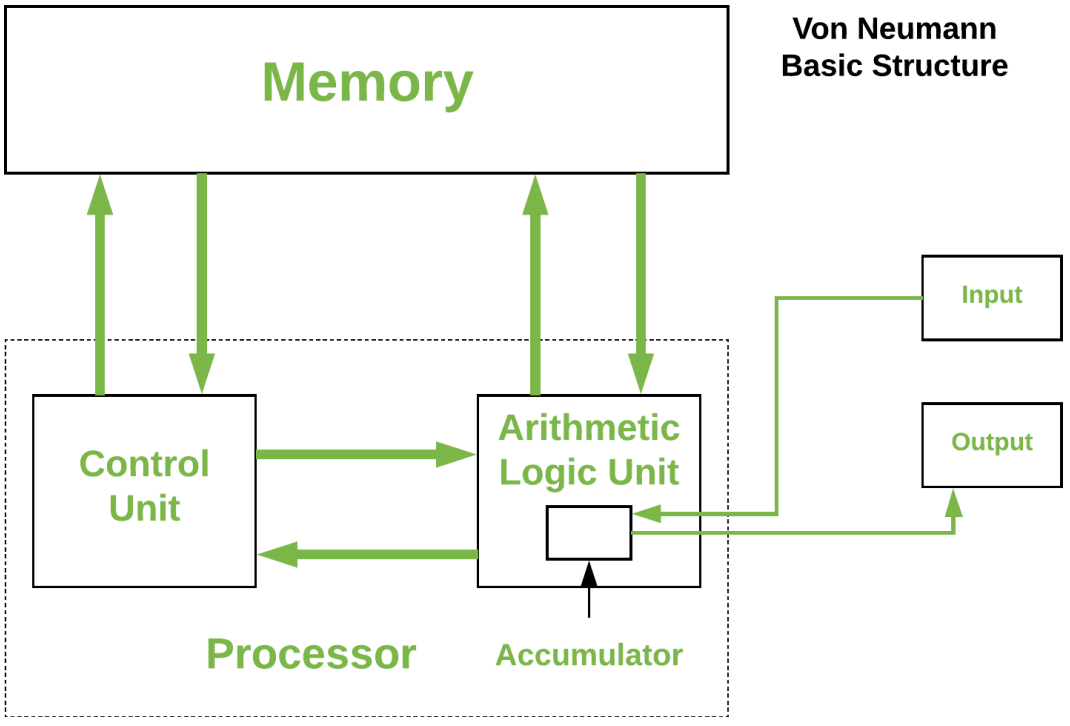
VON NEUMANN ARCHITECTURE

Computer Organization is like understanding the "blueprint" of how a computer works internally. One of the most important models in this field is the Von Neumann architecture, which is the foundation of most modern computers.
Named after John von Neumann, this architecture introduced the concept of storing both data and instructions in the same memory.

Historically there have been 2 types of Computers:

1. **Fixed Program Computers -** Their function is very specific and they couldn't be reprogrammed, e.g. Calculators.
2. **Stored Program Computers -** These can be programmed to carry out many different tasks, applications are stored on them, hence the name.

The Von Neumann architecture popularized the stored-program concept, making computers more flexible and easier to reprogram. This design stores both data and instructions in the same memory, simplifying hardware design and enabling general-purpose computing.



The structure described in the figure outlines the basic components of a computer system, particularly focusing on the memory and processor. Here's a breakdown of the components:

- **Memory:** This is where data and instructions are stored. It is a crucial part of the computer system that allows for the storage and retrieval of information.
- **Control Unit:** This component manages the operations of the computer. It directs the flow of data between the CPU and other components.
- **Arithmetic Logic Unit (ALU):** The ALU performs arithmetic and logical operations. It is responsible for calculations and decision-making processes.
- **Input:** This refers to the devices or methods through which data is entered into the computer system.
- **Output:** This refers to the devices or methods through which data is presented to the user or other systems.
- **Processor:** The processor, or CPU, is the central component that carries out the instructions of a computer program. It includes the ALU and Control Unit.
- **Accumulator:** This is a register in the CPU that stores intermediate results of arithmetic and logic operations.

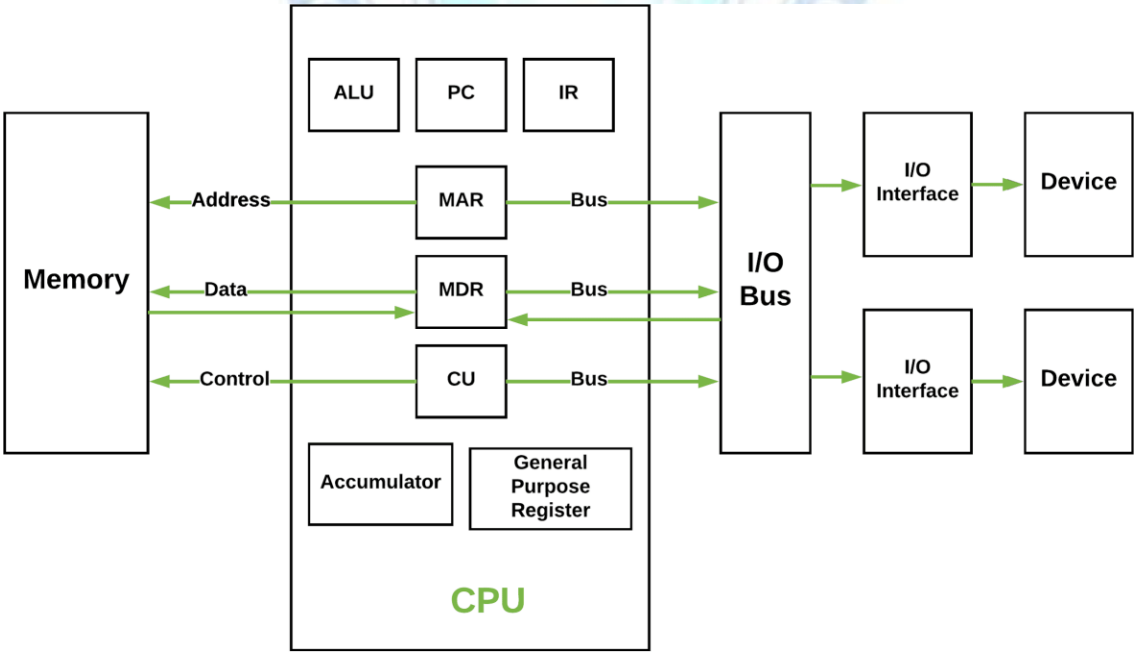


Figure - Basic CPU structure, illustrating ALU

The structure describes **Von Neumann Architecture**, which is a foundational design for modern computers. In this architecture, both data and instructions are

stored in the same memory and share a common bus for communication. Here's an explanation of the components of Von Neumann architecture:

Memory

- **Address**: Specifies the location in memory where data or instructions are stored or retrieved.
- **Data**: The actual information (either data or instructions) stored in memory.
- **Control**: Manages the flow of data and instructions between memory and the CPU.

CPU (Central Processing Unit)

The CPU is the core processing unit that executes instructions. It consists of:

- **ALU (Arithmetic Logic Unit)**: Performs arithmetic and logical operations (e.g., addition, subtraction, comparisons).
- **PC (Program Counter)**: Keeps track of the address of the next instruction to be executed.
- **IR (Instruction Register)**: Holds the current instruction being executed.
- **MAR (Memory Address Register)**: Stores the address of the memory location being accessed.
- **MDR (Memory Data Register)**: Temporarily holds data being transferred to or from memory.
- **CU (Control Unit)**: Coordinates the activities of the CPU, managing the flow of data and instructions.
- **Accumulator**: A register that stores intermediate results of arithmetic and logic operations.
- **General Purpose Registers**: Used for temporary storage of data during processing.

Bus

The bus is a communication system that transfers data, addresses, and control signals between the CPU, memory, and I/O devices. In Von Neumann architecture, a single bus is shared for both data and instructions, which can create a bottleneck (known as the Von Neumann bottleneck).

I/O Bus

- **I/O Interface**: Connects the CPU and memory to input/output devices.
- **Device**: Refers to external hardware like keyboards, monitors, or storage devices.

Key Characteristics of Von Neumann Architecture

1. **Single Memory for Data and Instructions**: Both data and program instructions are stored in the same memory.
2. **Shared Bus**: A single bus is used for transferring data, addresses, and control

signals, which can limit performance.

3. **Sequential Execution**: Instructions are executed one at a time in a sequential manner.

Von Neumann bottleneck

Whatever we do to enhance performance, we cannot get away from the fact that instructions can only be done one at a time and can only be carried out sequentially. Both of these factors hold back the competence of the CPU. This is commonly referred to as the 'Von Neumann bottleneck'. We can provide a Von Neumann processor with more cache, more RAM, or faster components but if original gains are to be made in CPU performance then an influential inspection needs to take place of CPU configuration.

Advantages of Von Neumann Architecture

- **Simplified Design**: Uses a single memory for data and instructions, reducing hardware complexity.
- **Cost-Effective**: Lower production costs due to fewer components.
- **Flexibility**: Can run various programs and makes it suitable for general-purpose computing.
- **Ease of Programming**: Unified memory structure simplifies software development.
- **Widely Adopted**: Forms the foundation of most modern computers hence, ensures widespread compatibility.

Limitations of Von Neumann Architecture

- **Memory Bottleneck**: Shared memory slows down data and instruction transfer.
- **Sequential Processing**: Cannot process data and instructions simultaneously.
- **Scalability Issues**: Struggles with high-performance tasks requiring rapid memory access.
- **Energy Inefficiency**: Frequent memory access increases power consumption.
- **Latency**: Data and instruction fetch delays reduce overall system efficiency.

Applications of Von Neumann Architecture

- **General-Purpose Computing**: Powers desktops, laptops, and smartphones.
- **Embedded Systems**: Used in simple devices where cost and simplicity are priorities.
- **Software Development**: Shapes programming tools and languages due to its unified

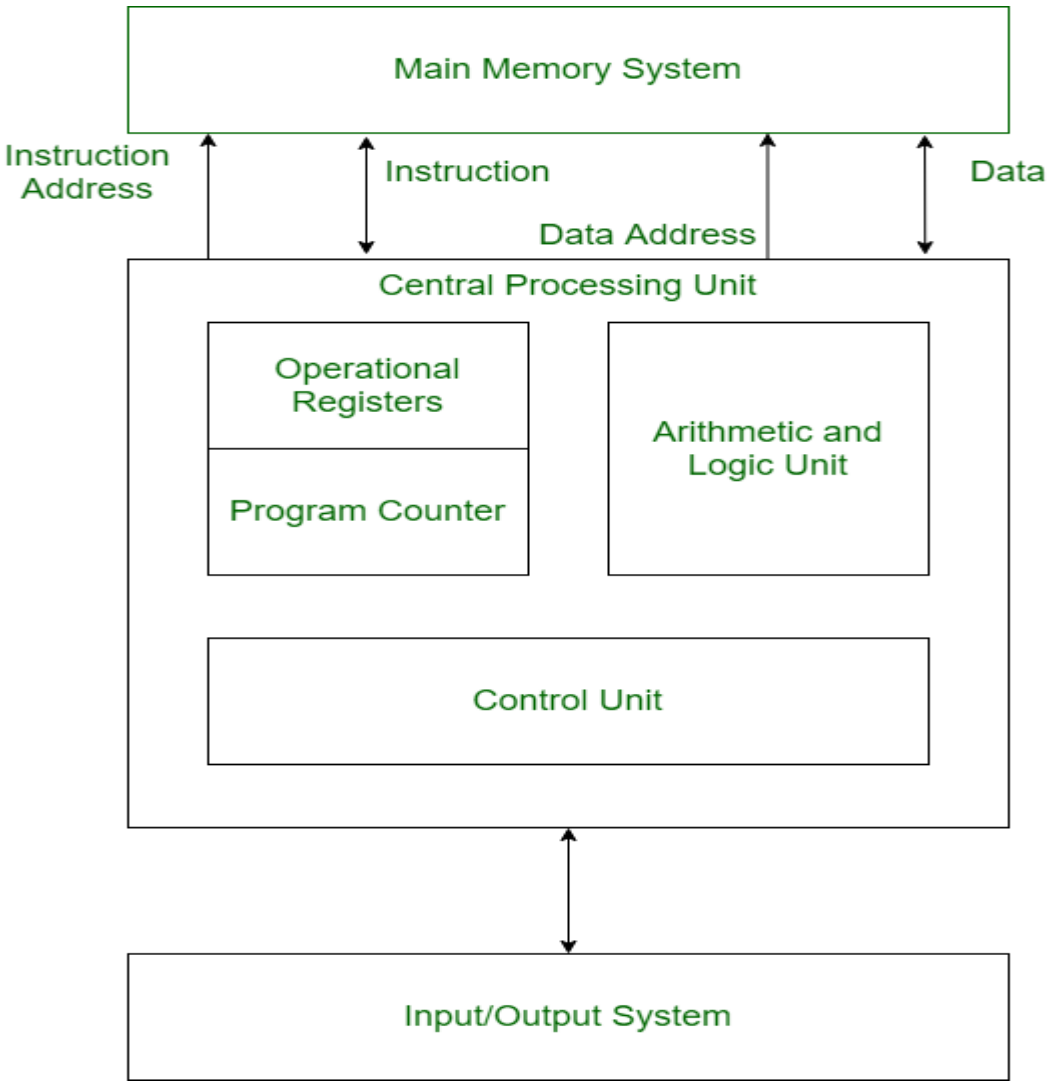
structure.

- **Education**: A foundational concept in computer science courses.
- **Gaming and Multimedia**: Supports complex applications like video games and editing software.

HARVARD ARCHITECTURE

In a normal computer that follows von Neumann architecture, instructions, and data both are stored in the same memory. So same buses are used to fetch instructions and data. This means the CPU cannot do both things together (read the instruction and read/write data). So, to overcome this problem, Harvard architecture was introduced.

The Harvard architecture is a computer architecture that separates memory storage and buses for instructions and data, unlike the von Neumann architecture, which uses a single memory and bus for both. This separation allows the CPU to access instructions and read/write data simultaneously, overcoming the bottleneck of von Neumann's architecture and enhancing processing speed. By enabling parallel access to instructions and data, the Harvard architecture improves performance, making it particularly suitable for applications requiring high-speed processing, such as embedded systems and digital signal processing.



Structure of Harvard Architecture

Buses:

Buses are used as signal pathways. In Harvard architecture, there are separate buses for both instruction and data. Types of Buses:

- **Data Bus:** It carries data among the main memory system, processor, and I/O devices.
- **Data Address Bus:** It carries the address of data from the processor to the main memory system.
- **Instruction Bus:** It carries instructions among the main memory system, processor, and I/O devices.
- **Instruction Address Bus:** It carries the address of instructions from the processor to the main memory system.

Operational Registers

There are different types of registers involved in it which are used for storing addresses of different types of instructions. For example, the Memory Address Register and Memory Data Register are operational registers.

- **Program Counter:** It has the location(address) of the next instruction to be executed. The program counter then passes this next address to the memory address register.
- **Arithmetic and Logic Unit:** The arithmetic logic unit is part of the CPU that operates all the calculations needed. It performs addition, subtraction, comparison, logical Operations, bit Shifting Operations, and various arithmetic operations.
- **Control Unit:** The Control Unit is the part of the CPU that operates all processor control signals. It controls the input and output devices and also controls the movement of instructions and data within the system.
- **Input / Output System:** Input devices are used to read data into main memory with the help of CPU input instruction. The information from a computer as output is given through Output devices. The computer gives the results of computation with the help of output devices.

Features of Harvard Architecture

- **Separate memory spaces:** Harvard architecture uses distinct memory spaces for instructions and data, enabling simultaneous access for faster processing.
- **Fixed instruction length:** In Harvard architecture, instructions are of fixed length, which simplifies the instruction fetch process and allows for faster instruction processing.
- **Parallel instruction and data access:** The separation of instruction and data memories allows parallel access, improving overall efficiency.
- **More efficient memory usage:** Harvard architecture allows for more efficient use of memory as the data and instruction memories can be optimized independently, which can lead to better performance.

- **Suitable for embedded systems:** Harvard architecture is commonly used in embedded systems because it provides fast and efficient access to both instructions and data, which is critical in real-time applications.
- **Limited flexibility:** The separate memory spaces restrict tasks like modifying instructions at runtime, reducing flexibility.

Advantage of Harvard Architecture

- **Fast and efficient data access:** Since Harvard architecture has separate memory spaces for instructions and data, it allows for parallel and simultaneous access to both memory spaces, which leads to faster and more efficient data access.
- **Better performance:** The use of fixed instruction length, parallel processing, and optimized memory usage in Harvard architecture can lead to improved performance and faster execution of instructions.
- **Suitable for real-time applications:** Harvard architecture is commonly used in embedded systems and other real-time applications where speed and efficiency are critical.
- **Security:** The separate spaces can also provide a degree of security against certain types of attacks, such as buffer overflow attacks.

Disadvantages of Harvard Architecture

- **Complexity:** The use of separate memory spaces for instructions and data in Harvard architecture adds to the complexity of the processor design and can increase the cost of manufacturing.
- **Limited flexibility:** Harvard architecture has limited flexibility in terms of modifying instructions at runtime because instructions and data are stored in separate memory spaces. This can make certain types of programming more difficult or impossible to implement.
- **Higher memory requirements:** Harvard architecture requires more memory than Von Neumann architecture , which can lead to higher costs and power consumption.
- **Code size limitations:** Fixed instruction length in Harvard architecture can limit the size of code that can be executed, making it unsuitable for some applications with larger code bases.

Difference between Von Neumann and Harvard Architecture

Von Neumann and Harvard architectures are the two basic models in the field of computer architecture, explaining the organization of memory and processing units in a computer system. For those involved in Computer Science or working in companies providing computing technologies, it is essential to understand the characteristics of these architectures.

There are two models of multiprocessing architectures: Von Neumann and Harvard. While the former occupies a dominant position, this article will discuss its principal differences from the latter, along with their respective advantages and disadvantages, to help you understand which architecture is more suitable for a given application.

VON NEUMANN ARCHITECTURE	HARVARD ARCHITECTURE
It is ancient computer architecture based on stored program computer concept.	It is modern computer architecture based on Harvard Mark I relay based model.
Same physical memory address is used for instructions and data.	Separate physical memory address is used for instructions and data.
There is common bus for data and instruction transfer.	Separate buses are used for transferring data and instruction.
Two clock cycles are required to execute single instruction.	An instruction is executed in a single cycle.
It is cheaper in cost.	It is costly than Von Neumann Architecture.
CPU cannot access instructions and read/write at the same time.	CPU can access instructions and read/write at the same time.
It is used in personal computers and small computers.	It is used in micro controllers and signal processing.
Suffers from the **Von Neumann bottleneck**, where the CPU waits for memory access.	Does not suffer significantly from the Von Neumann bottleneck.
Simpler memory organization	More complex memory organization.
Memory space can be shared dynamically between program and data.	Program and data memories are fixed and independent.
Lower performance due to shared memory access.	Higher performance due to parallel memory access.

EVOLUTION OF ISA

The evolution of ISA (Instruction Set Architecture) can be understood in two main contexts: computer architecture and the Instrument Society of America (now known as ISA, the automation and control association). In computer architecture, ISA refers to the abstract model of a computer that defines how the CPU is controlled by software. In the context of automation and control, ISA originated as the Instrument Society of America, founded in 1945, and has evolved to become a leading global organization for automation and control professionals.

Evolution of ISA in Computer Architecture:

- **Early ISAs:**

The earliest computers had very simple ISAs, often tied directly to the hardware.

- **The Rise of x86:**

The x86 ISA, initially developed by Intel, became dominant in personal computers and servers due to its backward compatibility and the vast ecosystem of software built around it.

- **RISC Architectures:**

Reduced Instruction Set Computing (RISC) architectures, like ARM, emerged as alternatives to the complex instruction set computing (CISC) of x86, offering advantages in power efficiency and performance for certain applications.

- **Evolution of ISA Features:**

ISAs have evolved to include features like:

  - **Variable-length instructions:** Allowing for more compact code but potentially increasing complexity.
  - **Advanced addressing modes:** Providing more flexible ways to access memory.
  - **Support for parallel processing:** Enabling faster execution of computations by dividing work among multiple processors.

- **Virtual Machines and JIT Compilation:**

Some ISAs, like those used in Java and .NET, are implemented on virtual machines, which translate bytecode into native machine code for efficient execution.

Evolution of ISA (the automation and control association):

- **Founding:**

ISA was founded in 1945 as the Instrument Society of America, with a focus on standardizing industrial instruments and sharing information.

- **Growth and Expansion:**

Over the decades, ISA has grown into a global organization with members from various industries, including automation, control, and manufacturing.

- **Focus on Standards and Education:**

ISA plays a crucial role in developing and promoting industry standards for automation and control systems. They also provide training and education to professionals in the field.

- **Digital Transformation:**

ISA has adapted to the digital age, with a focus on integrating digital technologies into automation and control systems.

Period	Development
1970s	Early processors used CISC architectures.
1980s	RISC was introduced to simplify processor design and improve speed.
1990s	x86 became the dominant architecture for PCs.
2000s	ARM dominated mobile and embedded devices because of low power consumption.
Today	x86 powers many desktops and servers, while ARM dominates smartphones and is increasingly used in laptops, servers, and cloud computing.

RISC and CISC in Computer Organization

RISC is the way to make hardware simpler whereas CISC is the single instruction that handles multiple work. In this article, we are going to discuss RISC and CISC in detail as well as the Difference between RISC and CISC, Let's proceed with RISC first.

Reduced Instruction Set Architecture (RISC)

The main idea behind this is to simplify hardware by using an instruction set composed of a few basic steps for loading, evaluating, and storing operations just like a load command will load data, a store command will store the data.

Characteristics of RISC

- Simpler instruction, hence simple instruction decoding.
- Instruction comes undersize of one word.
- Instruction takes a single clock cycle to get executed.
- More general-purpose registers.
- Simple Addressing Modes.
- Fewer Data types.

- A pipeline can be achieved.

Advantages of RISC

- **Simpler instructions:** RISC processors use a smaller set of simple instructions, which makes them easier to decode and execute quickly. This results in faster processing times.
- **Faster execution:** Because RISC processors have a simpler instruction set, they can execute instructions faster than CISC processors.
- **Lower power consumption:** RISC processors consume less power than CISC processors, making them ideal for portable devices.

Disadvantages of RISC

- **More instructions required:** RISC processors require more instructions to perform complex tasks than CISC processors.
- **Increased memory usage:** RISC processors require more memory to store the additional instructions needed to perform complex tasks.
- **Higher cost:** Developing and manufacturing RISC processors can be more expensive than CISC processors.

Complex Instruction Set Architecture (CISC)

The main idea is that a single instruction will do all loading, evaluating, and storing operations just like a multiplication command will do stuff like loading data, evaluating, and storing it, hence it's complex.

Characteristics of CISC

- Complex instruction, hence complex instruction decoding.
- Instructions are larger than one-word size.
- Instruction may take more than a single clock cycle to get executed.
- Less number of general-purpose registers as operations get performed in memory itself.
- Many instructions access memory directly.
- Complex/multiple Addressing Modes.
- More Data types.
- Smaller program size.
- More hardware complexity.

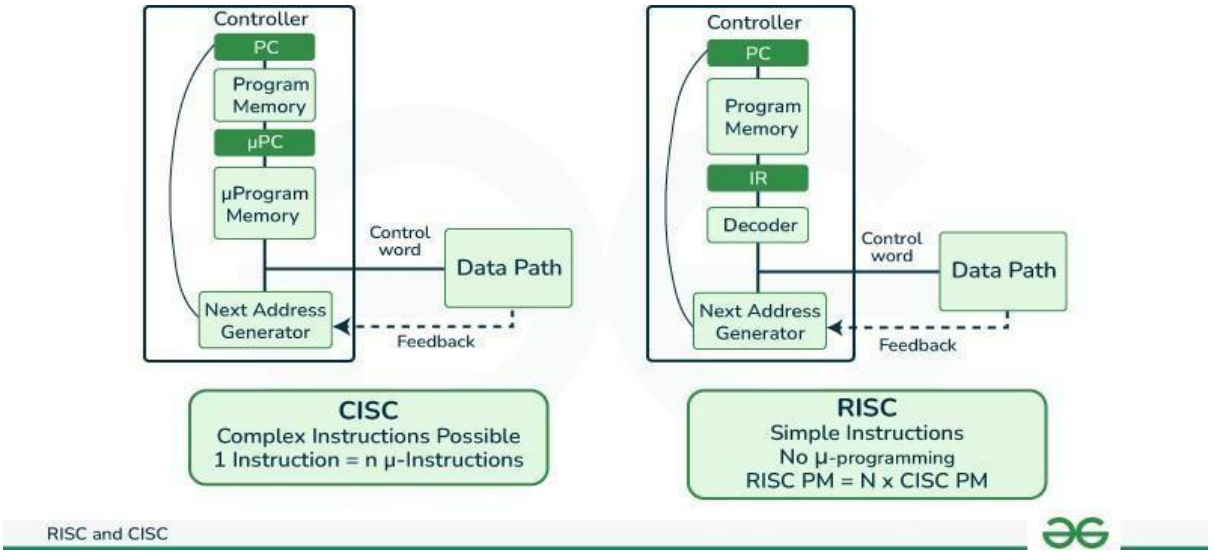
Advantages of CISC

- **Reduced code size:** CISC processors use complex instructions that can perform multiple operations, reducing the amount of code needed to perform a task.
- **More memory efficient:** Because CISC instructions are more complex, they require fewer instructions to perform complex tasks, which can result in more memory-efficient code.

- **Widely used:** CISC processors have been in use for a longer time than RISC processors, so they have a larger user base and more available software.

Disadvantages of CISC

- **Slower execution:** CISC processors take longer to execute instructions because they have more complex instructions and need more time to decode them.
- **More complex design:** CISC processors have more complex instruction sets, which makes them more difficult to design and manufacture.
- **Higher power consumption:** CISC processors consume more power than RISC processors because of their more complex instruction sets.



CPU Performance

Both approaches try to increase the CPU performance

- **RISC:** Reduce the cycles per instruction at the cost of the number of instructions per program.

$$CPU\ Time = \frac{Seconds}{Program} = \frac{Instructions}{Program} X \frac{Cycles}{Instructions} X \frac{Seconds}{Cycle}$$

CPU Time

- **CISC:** The CISC approach attempts to minimize the number of instructions per program but at the cost of an increase in the number of cycles per instruction.

Earlier when programming was done using assembly language, a need was felt to make instruction do more tasks because programming in assembly was tedious and error-prone due to which CISC architecture evolved but with the uprise of high-level language dependency on assembly reduced RISC architecture prevailed.

Example:

Suppose we have to add two 8-bit numbers:

- **CISC approach:** There will be a single command or instruction for this like ADD which will perform the task.
- **RISC approach:** Here programmer will write the first load command to load data in registers then it will use a suitable operator and then it will store the result in the desired location.

So, add operation is divided into parts i.e. load, operate, store due to which RISC programs are longer and require more memory to get stored but require fewer transistors due to less complex command.

RISC vs CISC

RISC	CISC
Focus on software	Focus on hardware
Uses only Hardwired control unit	Uses both hardwired and microprogrammed control unit
Transistors are used for more registers	Transistors are used for storing complex Instructions
Fixed sized instructions	Variable sized instructions
Can perform only Register to Register Arithmetic operations	Can perform REG to REG or REG to MEM or MEM to MEM
Requires more number of registers	Requires less number of registers
Code size is large	Code size is small
An instruction executed in a single clock cycle	Instruction takes more than one clock cycle
An instruction fit in one word.	Instructions are larger than the size of one word
Simple and limited addressing modes.	Complex and more addressing modes.
RISC is Reduced Instruction Cycle.	CISC is Complex Instruction Cycle.
The number of instructions are less as compared to CISC.	The number of instructions are more as compared to RISC.
It consumes the low power.	It consumes more/high power.

RISC is highly pipelined.	CISC is less pipelined.
RISC required more RAM .	CISC required less RAM.
Here, Addressing modes are less.	Here, Addressing modes are more.

ARM processor

Introduction

ARM (Advanced RISC Machine) is a family of processors based on the **RISC (Reduced Instruction Set Computing)** architecture.

It was originally developed by **Acorn Computers** in the 1980s. Today, ARM processors are designed by **Arm Holdings**, which licenses the processor designs to many companies instead of manufacturing the chips itself.

Companies such as **Apple, Qualcomm, Samsung, MediaTek, and NVIDIA** use ARM designs in their processors.

ARM(Definition)

- ARM is a **RISC-based processor architecture**.
- It is known for **high performance** and **low power consumption**.
- ARM does **not manufacture processors**; instead, it **licenses its designs** to other companies.
- Companies customize ARM processors according to their product requirements.
- ARM processors are widely used in **mobile devices, embedded systems, IoT devices, and wearable devices**.

ARM Business Model

ARM follows a **licensing business model**.

- Arm Holdings designs the processor architecture.
- Other companies purchase a license to use or modify the design.
- These companies manufacture their own ARM-based processors.

This allows manufacturers to develop processors for different types of devices while reducing design costs.

ARM processors are popular because they provide:

- Low power consumption
- High performance
- Longer battery life
- Small chip size
- Lower heat generation
- Cost-effective processor design

Because of these advantages, ARM processors are the preferred choice for smartphones, tablets, and other portable devices.

Common ARM Processor Families

- **Cortex-M Series** : For microcontrollers (low power, real-time control)
- **Cortex-R Series** : For real-time systems (e.g., automotive, robotics)
- **Cortex-A Series** : For application processors (e.g., smartphones, tablets)
- **Neoverse** : For infrastructure and cloud computing.
- **Apple Silicon (e.g., M1, M2)** : Custom ARM-based processors for Macs.

ARM assembly instructions are generally:

- Fixed length (32-bit or 16-bit Thumb)
- Simple
- Execute in fewer clock cycles
- Load/Store architecture

ARM Registers

Register	Purpose
R0–R12	General Purpose Registers
SP	Stack Pointer
LR	Link Register
PC	Program Counter

Common ARM Instructions

Instruction	Function
MOV	Move data
ADD	Addition
SUB	Subtraction
MUL	Multiplication
CMP	Compare
B	Branch
BL	Branch with Link
LDR	Load data
STR	Store data

ARM Program Example

```
MOV R0,#15
MOV R1,#25
ADD R2,R0,R1
```

Explanation

- Load 15 into R0.
- Load 25 into R1.
- Add R0 and R1.
- Store result in R2.

Output:

R2 = 40

Advantages of ARM processor

- **Low Power Consumption**: Ideal for battery-powered and mobile devices.
- **High Performance per Watt**: Efficient processing with minimal energy use.
- **Compact and Simple Design**: Reduces manufacturing costs and chip size.
- **RISC Architecture**: Simplifies instructions, enabling faster execution.
- **Wide Ecosystem Support**: Extensive software and development tools.
- **Scalability**: Used in a variety of devices from microcontrollers to smartphones.
- **Cost-Effective Licensing Model** : Enables broad adoption by manufacturers.

Disadvantages of ARM processor

- **Incompatible with x86 Systems**: They cannot natively run x86 based software, limiting compatibility with Windows systems.
- **Limited High-End Performance**: ARM processors generally offer lower performance compared to high end x86 CPUs.
- **Requires Skilled Programming** : Programming for ARM can be complex and requires experienced developers.
- **Less Efficient Instruction Scheduling** : ARM is less efficient in handling instruction scheduling, which may affect performance in complex tasks.

X86 PROCESSOR

Introduction

x86 is a family of processors based on the **CISC (Complex Instruction Set Computing)** architecture. It was originally developed by Intel and is now also used by AMD.

Unlike ARM, companies such as Intel and AMD **both design and manufacture** their own x86 processors.

x86 Processor (DEFINITION)

- x86 is a **CISC-based processor architecture**.
- It is designed for **high performance and powerful computing**.
- Intel and AMD **design and manufacture** their own processors.
- x86 processors are widely used in **desktop computers, laptops, workstations, and servers**.

x86 Business Model

Unlike ARM, x86 processors are **not licensed to other companies**.

- Intel designs and manufactures Intel Core processors.
- AMD designs and manufactures Ryzen and EPYC processors.

Other companies generally cannot manufacture x86 processors because the x86 instruction set is owned and controlled mainly by Intel and AMD through licensing agreements.

x86 processors are popular because they provide:

- High processing performance
- Excellent multitasking
- Support for complex applications
- Compatibility with older software (backward compatibility)
- Large software ecosystem

x86 Assembly Language

The x86 processor uses **CISC (Complex Instruction Set Computing)** architecture.

It supports

- Complex instructions
- Variable-length instructions
- Multiple addressing modes

x86 Registers

Register	Purpose
AX	Accumulator
BX	Base Register
CX	Counter Register
DX	Data Register
SP	Stack Pointer
BP	Base Pointer
SI	Source Index
DI	Destination Index
IP	Instruction Pointer

Common x86 Instructions

Instruction	Function
MOV	Transfer data
ADD	Addition
SUB	Subtraction
MUL	Multiplication
DIV	Division
CMP	Compare
JMP	Jump
INC	Increment
DEC	Decrement

x86 Program Example

```
MOV AX,10
MOV BX,20
ADD AX,BX
```

Explanation

- Load 10 into AX.
- Load 20 into BX.
- Add BX to AX.

Output

AX = 30

Difference between ARM and x86

ARM	x86
ARM uses Reduced Instruction Set Computing Architecture (RISC).	x86 uses Complex Instruction Set Architecture (CISC).
ARM works by executing single instruction per cycle.	It works by executing complex instructions at once and it requires more than one cycle.
Performance can be optimized by a Software-based approach.	Performance can be optimized by Hardware based approach.
It require fewer registers, but they require more memory.	It processors require less memory, but more registers.
Execution is faster in ARM Processes.	Execution is slower in an x86 Processor.
They use the memory which is already available to them.	They require some extra memory for calculations.
They are deployed in mobiles which deal with the consumption of power, speed, and size.	They are deployed in Servers, Laptops where performance and stability matter.
ARM Processor work by generating multiple instructions from a complex instruction and they are executed separately.	x86 Processors work by executing complex statements at a single time.

COMPONENTS OF A COMPUTER

The functional components of a digital computer include the Input Unit, which takes in data; the CPU, which processes data with its Control Unit (CU), Arithmetic Logic Unit (ALU), and Registers; the Memory Unit, which stores data temporarily (RAM) or permanently (HDD/SSD); the Output Unit, which displays results; and the Bus System, which connects and transfers data between components. These parts work together to execute tasks and provide results.

Functional Components of Computer

The functional components of a computer are the key parts that work together to process and manage data. These include the Input Unit for receiving data, the CPU for processing it, the Memory Unit for storing information,
the Output Unit for displaying results, and the Bus System that connects all parts. These components help the computer perform tasks efficiently.

1. Input Unit

- **Purpose:** Captures data and instructions from users or external sources.
- **Function:** Converts user input into binary signals that the computer can process.
- **Common Devices (2025):**
  - Keyboard, Mouse, Touchscreens
  - Scanners, Sensors, Stylus pens
  - Voice Assistants (e.g., Siri, Alexa)
  - Biometric devices (face/fingerprint recognition)
  - Iot-based inputs from smart devices

2. Central Processing Unit (CPU) – The Brain of the Computer

The CPU executes instructions and controls all internal operations. In 2025, CPUs will often have multiple cores and threads to handle parallel processing efficiently.

Components of CPU:

a. Arithmetic Logic Unit (ALU)

- Performs arithmetic operations (add, subtract, multiply, divide).
- Handles logical operations (comparison, decision-making).
- Supports AI/ML tasks using built-in vector/matrix operations (in modern CPUs).

b. Control Unit (CU)

- Directs the operations of all computer parts.
- Decodes instructions and coordinates data flow.
- Sends control signals to memory and I/O devices.

c. Registers

- **High-speed memory locations** within the CPU.
- Temporarily store instructions, addresses, and intermediate data.
- Examples: Accumulator, Instruction Register, Program Counter, Address Register.
- **Modern CPUs** include 64-bit or even 128-bit registers for faster processing.

3. Memory / Storage Unit

The **memory unit** holds data and instructions before, during, and after processing.

a. Primary Memory (Main Memory):

- **RAM (Random Access Memory):** Temporarily stores data during execution.
- Types in 2025: DDR5, LPDDR5X, and emerging **MRAM**.
- **ROM (Read-Only Memory):** Stores boot-up instructions and firmware.
- **Cache Memory:** Ultra-fast memory between CPU and RAM (L1, L2, L3 levels).

b. Secondary Storage:

- Used for long-term data storage.
- **Examples**: SSDs (NVMe drives), HDDs, flash drives, and cloud storage.
- **Modern Trend:** Use of Cloud Integration and hybrid storage models.

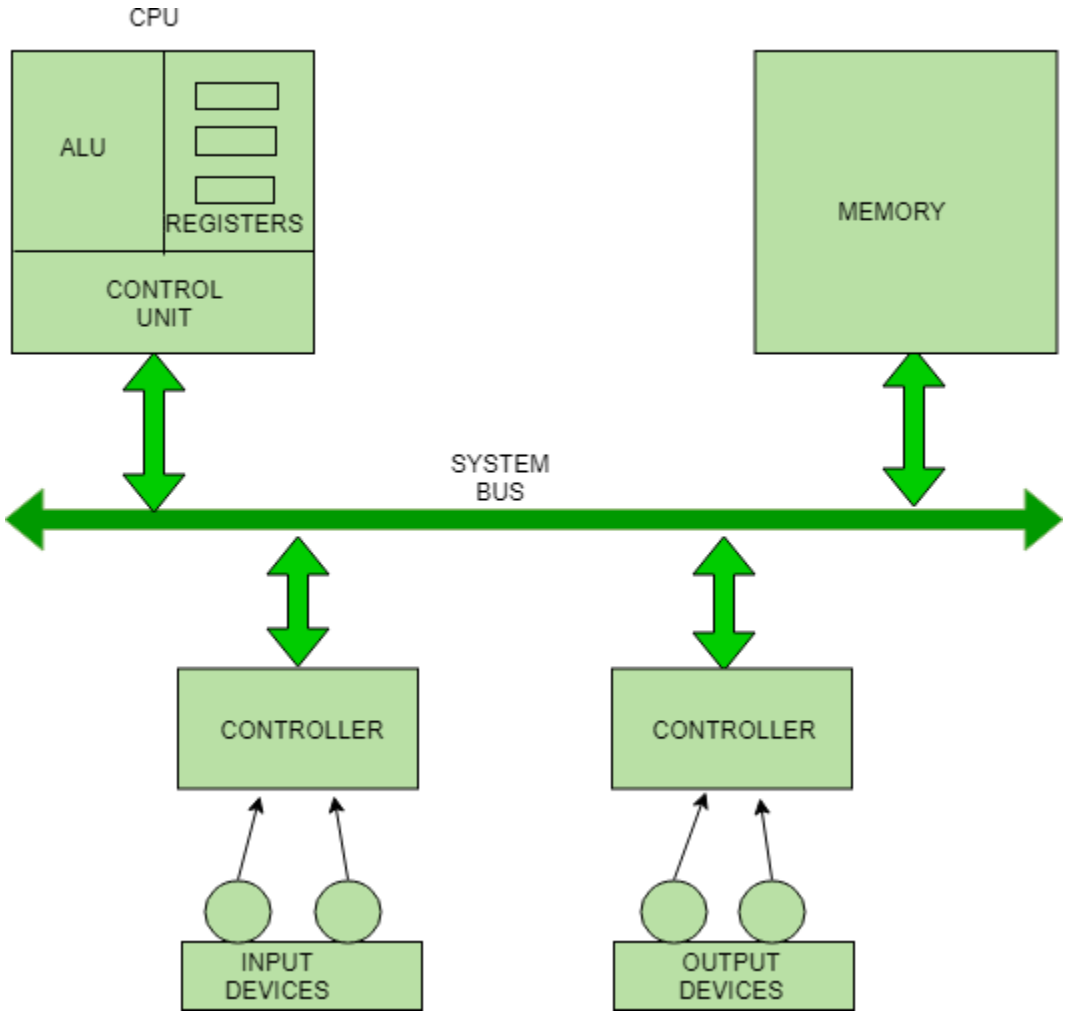
4. Output Unit

- **Purpose:** Converts processed data (binary) into a form users can understand.
- **Examples:**
  - Visual: Monitors (LED, OLED, 4K/8K displays)
  - Print: Printers (Inkjet, Laser, 3D Printers)
  - Audio: Speakers, Headphones
  - Haptic: Vibration feedback devices
- **Emerging Tech:** AR/VR headsets, voice-based output, Braille displays for accessibility

Interconnection between Functional Components

A computer consists of an input unit that takes input, a CPU that processes the input and an output unit that produces output. All these devices communicate with each other through a common bus. A bus is a transmission path, made of a set of conducting wires over which data or information in the form of electric signals is passed from one component to another in a computer. The bus can be of three types – Address bus, Data bus and Control Bus.

The following figure shows the connection of various functional components:



The address bus carries the address location of the data or instruction. The data bus carries data from one component to another, and the control bus carries the control signals. The system bus is the common communication path that carries signals to/from the CPU, main memory and input/output devices. The input/output devices communicate with the system bus through the controller circuit, which helps in managing various input/output devices attached to the computer.

Conclusion

The **functional components of a computer** work in unison to process data and perform tasks efficiently. Each component, from the input and output units to the CPU, memory, and bus system, has a crucial role in handling information, carrying out calculations, and delivering results. Understanding how these components interact helps us appreciate how a computer operates and how its hardware and software collaborate to perform complex functions.

Addressing modes

Addressing modes are the techniques used by the CPU to identify where the data needed for an operation is stored. They provide rules for interpreting or modifying the address field in an instruction before accessing the operand.

Addressing modes for 8086 instructions are divided into two categories:

1) Addressing modes for data

2) Addressing modes for branch

The 8086 memory addressing modes provide flexible access to memory, allowing us to easily access variables, arrays, records, pointers, and other complex data types. The key to good assembly language programming is the proper use of memory addressing modes.

An assembly language program instruction consists of two parts

Opcode	Operand

The memory address of an operand consists of two components:

IMPORTANT TERMS

- **Starting address** of memory segment.
- **Effective address or Offset**: An offset is determined by adding any combination of three address elements: displacement, base and index.
    - **Displacement:** It is an 8 bit or 16 bit immediate value given in the instruction.
    - **Base**: Contents of base register, BX (Base Register)or BP (Base Pointer Register).
    - **Index**: Content of index register SI (Source Index Register) or DI (Destination Index Register).

According to different ways of specifying an operand by 8086 microprocessor, different addressing modes are used by 8086.

Importance of Addressing Modes

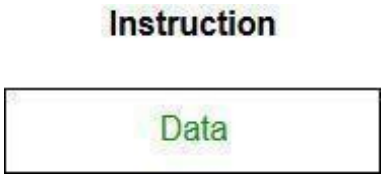
- They allow flexibility in data handling, such as accessing arrays, records, or pointers.
- They support program control with techniques like loops, branches, and jumps.
- They enable efficient memory usage and program relocation during runtime.
- They reduce the complexity of programming by offering multiple ways to access data.

Types of Addressing Modes in Computer Architecture

Addressing Modes used by 8086 microprocessor are discussed below:

Implied mode

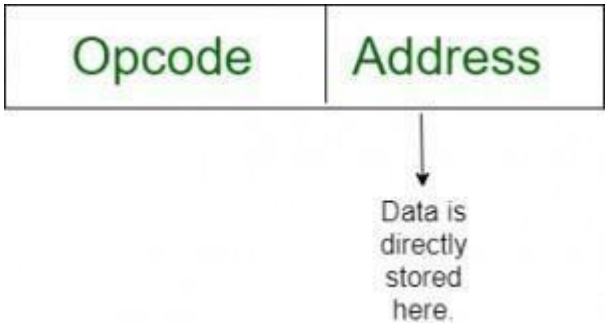
In implied addressing the operand is specified in the instruction itself. In this mode the data is 8 bits or 16 bits long and data is the part of instruction. Zero address instruction are designed with implied addressing mode.



Example: CLC
(used to reset Carry flag to 0)

Immediate addressing mode (symbol #)

In this mode data is present in address field of instruction .Designed like one address instruction format.
Note: Limitation in the immediate mode is that the range of constants are restricted by size of address field.

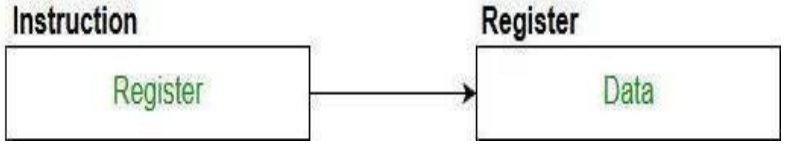


Example: MOV AL, 35H
(move the data 35H into AL register)

Register mode

In register addressing the operand is placed in one of 8 bit or 16 bit general purpose registers. The data is in the register that is specified by the instruction.

Here one register reference is required to access the data.



Example: MOV AX,CX
(move the contents of CX register to AX register)

Register Indirect mode

In this addressing the operand's offset is placed in any one of the registers BX,BP,SI,DI as specified in the instruction. The effective address of the data is in the base register or an index register that is specified by the instruction. Here two register reference is required to access the data.

The 8086 CPUs let you access memory indirectly through a register using the register indirect addressing modes.

MOV AX, [BX]
(move the contents of memory location addressed by the register BX to the register AX)

Auto Indexed (increment mode)

Effective address of the operand is the contents of a register specified in the instruction. After accessing the operand, the contents of this register are automatically incremented to point to the next consecutive memory location.**(R1)+**. Here one register reference, one memory reference and one ALU operation is required to access the data. Example:

Add R1, (R2)+ // OR R1 = R1 +M[R2]
R2 = R2 + d

Useful for stepping through arrays in a loop. R2 - start of array d - size of an element

Auto indexed (decrement mode)

Effective address of the operand is the contents of a register specified in the instruction. Before accessing the operand, the contents of this register are automatically decremented to point to the previous consecutive memory location. **-(R1)**Here one register reference, one memory reference and one ALU operation is required to access the data. Example:

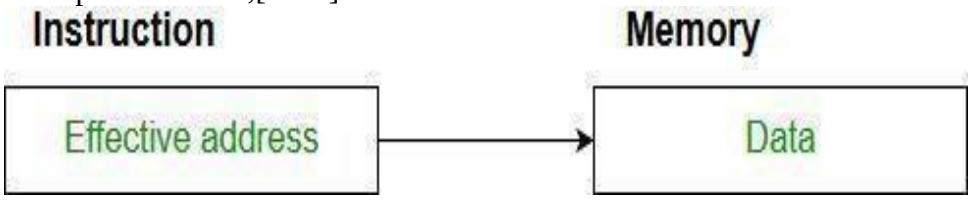
Add R1,-(R2) //OR R2 = R2-d
R1 = R1 + M[R2]

Auto decrement mode is same as auto increment mode. Both can also be used to implement a stack as push and pop . Auto increment and Auto decrement modes are useful for implementing "Last-In-First-Out" data structures.

Direct addressing/ Absolute addressing Mode (symbol [ ])

The operand's offset is given in the instruction as an 8 bit or 16 bit displacement element. In this addressing mode the 16 bit effective address of the data is the part of the instruction. Here only one memory reference operation is required to access the data.

Example: ADD AL,[0301] //add the contents of offset address 0301 to AL



Indirect addressing Mode (symbol @ or ())

In this mode address field of instruction contains the address of effective address. Here two references are required. 1st reference to get effective address. 2nd reference to access the data. Based on the availability of Effective address, Indirect mode is of two kind:

- **Register Indirect:** In this mode effective address is in the register, and corresponding register name will be maintained in the address field of an instruction. Here one register reference, one memory reference is required to access the data.

Example : MOV A, @R0

- **Memory Indirect**: In this mode effective address is in the memory, and corresponding memory address will be maintained in the address field of an instruction. Here two memory reference is required to access the data.

Example : MOV AX, [[5000H]]

Indexed addressing mode

The operand's offset is the sum of the content of an index register SI or DI and an 8 bit or 16 bit displacement.

Example: MOV AX, [SI +05]

Based Indexed Addressing

The operand's offset is sum of the content of a base register BX and an index register SI or DI. Example:

ADD AX, [BX+SI]

Based on Transfer of control, addressing modes are:

PC relative addressing mode

PC relative addressing mode is used to implement intra segment transfer of control, In this mode effective address is obtained by adding displacement to PC.

EA= PC + Address field value PC= PC + Relative value.

Base register addressing mode

Base register addressing mode is used to implement inter segment transfer of control. In this mode effective address is obtained by adding base register value to address field value.

EA= Base register + Address field value. PC= Base register + Relative value

Note :

- PC relative and based register both addressing modes are suitable for program relocation at runtime.
- Based register addressing mode is best suitable to write position independent codes.

Advantages of Addressing Modes

- Enable advanced programming techniques like pointers and counters for loops.
- Simplify memory access for arrays and complex data structures.
- Allow program relocation during runtime.

- Reduce the size of the instruction field, making the program more efficient.

Sample Question:

A robotics manufacturing company is designing an **autonomous robotic arm** that must perform precise pick-and-place operations. The embedded controller for the robot uses assembly language instructions with multiple addressing modes (immediate, direct, indirect, register, indexed, and base-relative). As the lead embedded systems engineer, explain how each addressing mode can be applied in the robot's control program to optimize **speed, memory usage, and flexibility**. Provide examples of instruction usage for each addressing mode and justify why certain modes are better suited for specific robotic tasks.

1. Immediate Addressing Mode

- **Definition:** Operand is part of the instruction itself.
- **Usage in Robotics:** Ideal for constants such as thresholds, speed limits, or actuator offsets.

Example:

- MOV R1, #5 ; Load constant 5 (e.g., fixed delay cycles for gripping)

2. Direct Addressing Mode

- **Definition:** The instruction specifies the memory address of the operand.
- **Usage in Robotics:** Reading fixed sensor registers or actuator control registers.

Example:

- MOV R2, [1000H] ; Load data from memory address 1000H (e.g., IR sensor value)

3. Indirect Addressing Mode

- **Definition:** Operand's memory address is stored in a register.
- **Usage in Robotics:** Handling variable sensor arrays (e.g., multiple joint encoders).

Example:

MOV R3, [R5] ; Load value from memory location stored in R5 (R5 points to encoder data)

4. Register Addressing Mode

- **Definition:** Operand is in a CPU register.
- **Usage in Robotics:** Intermediate position calculations, arithmetic, and loop counters.

Example:

ADD R1, R2 ; Add joint offset in R2 to current angle in R1

5. Indexed Addressing Mode

- **Definition:** Operand's address is computed as base + index.
- **Usage in Robotics:** Accessing lookup tables (e.g., precomputed sine/cosine for trajectory planning).

Example:

```
MOV R4, [BASE + R6]        ; Get trajectory point from lookup table using index R6
```

Base-Relative Addressing Mode

- **Definition:** Operand's address = base register + displacement.
- **Usage in Robotics**: Managing task-specific data blocks (e.g., grip parameters for different objects).

Example:

```
MOV R7, [BP + 4]        ; Load offset data for gripper force from stack frame
```

Instruction Formats

An Instruction Format defines the layout of bits in an instruction, i.e., how opcode, addressing mode, registers, and operands are represented in binary.

Common fields in instruction formats:

- **Opcode** → Specifies operation (ADD, MOV, JMP, etc.)
- **Addressing mode bits** → Indicate how operand is fetched
- **Register(s)** → Specify CPU registers involved
- **Displacement / Immediate field** → Holds address offset or constant Types

of Instruction Formats

1. **Zero-address format (Stack-based)**
   - Operands are implicitly on top of stack.
   - Example:
   - ADD ; Pops two values from stack, adds, pushes result
2. **One-address format (Accumulator-based)**
   - One operand in instruction, the other in accumulator.
   - Example:
   - ADD X ; ACC ← ACC + M[X]
3. **Two-address format**

- Specifies two operands explicitly.
- Example:
- ADD R1, R2 ; R1 ← R1 + R2

4. **Three-address format**
   - Specifies three operands.
   - Example:
   - ADD R1, R2, R3 ; R1 ← R2 + R3

Explain different instruction formats with neat diagrams. Give suitable examples for Zero, One, Two, and Three address formats. Compare their advantages and disadvantages.

1. Zero Address Format

- Used in **stack-based architectures** (e.g., JVM).
- **Diagram:** [Opcode]

Example:

```
PUSH A
PUSH B
ADD
```

- **Advantage:** Simple, less bits per instruction.
- **Disadvantage:** More instructions needed for complex operations.

2. One Address Format

- Uses **accumulator (ACC)** as implicit operand.
- **Diagram:** [Opcode | Address]
- **Example:**

```
LOAD A      ; ACC ← M[A]
ADD B       ; ACC ← ACC + M[B]
STORE C     ; M[C] ← ACC
```

- **Advantage:** Shorter instructions, efficient for simple CPUs.
- **Disadvantage:** High memory traffic (ACC is frequently used).

3. Two Address Format

- Two explicit operands, one also acts as destination.
- **Diagram:** [Opcode | Reg1 | Reg2]

Example:

- MOV R1, A
- ADD R1, B ; R1 ← R1 + B
- **Advantage:** Fewer instructions than 1-address.
- **Disadvantage:** One operand gets overwritten.

4. Three Address Format

- All operands specified explicitly.
- **Diagram:** [Opcode | Reg1 | Reg2 | Reg3]

Example:

- ADD R1, R2, R3 ; R1 ← R2 + R3
- **Advantage:** Very powerful, fewer instructions for complex tasks.
- **Disadvantage:** Instruction length increases (more bits needed).

Assembly Programming Basics (ARM/x86)

Introduction

Assembly language is a **low-level programming language** that is closely related to machine language. It uses **mnemonic codes** (such as MOV, ADD, SUB) instead of binary numbers, making programming easier while still allowing direct control of the processor.

Each processor architecture has its own assembly language.

- **ARM Assembly** - Used in ARM processors (RISC architecture).
- **x86 Assembly** - Used in Intel/AMD processors (CISC architecture).

Assembly language is converted into machine language using an **Assembler**.

Features of Assembly Language

- Low-level language
- Hardware dependent
- Fast execution speed
- Efficient memory usage
- Direct hardware control
- Uses mnemonic instructions

ARM Assembly Language

Introduction

ARM (Advanced RISC Machine) processors use **RISC (Reduced Instruction Set Computing)** architecture.

ARM assembly instructions are generally:

- Fixed length (32-bit or 16-bit Thumb)
- Simple
- Execute in fewer clock cycles
- Load/Store architecture

ARM Registers

Register	Purpose
R0–R12	General Purpose Registers
SP	Stack Pointer
LR	Link Register
PC	Program Counter

Common ARM Instructions

Instruction	Function
MOV	Move data
ADD	Addition
SUB	Subtraction
MUL	Multiplication
CMP	Compare
B	Branch
BL	Branch with Link
LDR	Load data
STR	Store data

ARM Program Example

```
MOV R0,#15
MOV R1,#25
ADD R2,R0,R1
```

Explanation

- Load 15 into R0.
- Load 25 into R1.
- Add R0 and R1.
- Store result in R2.

Output:

R2 = 40

x86 Assembly Language

The x86 processor uses **CISC (Complex Instruction Set Computing)** architecture.

It supports

- Complex instructions
- Variable-length instructions
- Multiple addressing modes

x86 Registers

Register	Purpose
AX	Accumulator
BX	Base Register
CX	Counter Register
DX	Data Register
SP	Stack Pointer
BP	Base Pointer
SI	Source Index
DI	Destination Index
IP	Instruction Pointer

Common x86 Instructions

Instruction	Function
MOV	Transfer data
ADD	Addition
SUB	Subtraction
MUL	Multiplication
DIV	Division
CMP	Compare
JMP	Jump
INC	Increment
DEC	Decrement

x86 Program Example

```
MOV AX,10
MOV BX,20
ADD AX,BX
```

Explanation

- Load 10 into AX.
- Load 20 into BX.
- Add BX to AX.

Output

AX = 30

ARM vs x86 Assembly

ARM	x86
RISC Architecture	CISC Architecture
Fixed-length instructions	Variable-length instructions
More registers	Fewer registers
Simple instructions	Complex instructions
Load/Store architecture	Memory operations allowed
Lower power consumption	Higher power consumption

PERFORMANCE METRICS

Performance metrics are used to measure how efficiently a processor executes a program.

The most common performance metrics are:

- CPU Execution Time
- CPI (Cycles Per Instruction)
- MIPS (Million Instructions Per Second)
- Amdahl's Law

CPU Execution Time

CPU execution time is the total time required by the processor to execute a program.

Formula

$$CPU\ Time = Instruction\ Count \times CPI \times Clock\ Cycle\ Time$$

where

- Instruction Count = Number of instructions executed
- CPI = Cycles per instruction
- Clock Cycle Time = Time for one clock cycle

Example

Instruction Count = 2,000,000

CPI = 2

Clock Cycle Time = 0.5 ns

CPU Time

$$= 2,000,000 \times 2 \times 0.5ns = 2ms$$

CPI (Cycles Per Instruction)

CPI is the average number of clock cycles required to execute one instruction.

Formula

$$CPI = \frac{Total\ Clock\ Cycles}{Total\ Instructions}$$

Example

Clock Cycles = 1000

Instructions = 500

$$CPI = \frac{1000}{500} = 2$$

Meaning:

Each instruction requires **2 clock cycles**.

Factors Affecting CPI

- Cache memory

- Pipeline stalls
- Branch prediction
- Instruction complexity
- Memory access delay

2. MIPS (Million Instructions Per Second)

MIPS measures processor speed.

Formula

$$MIPS = \frac{Instruction\ Count}{Execution\ Time \times 10^6}$$

Example

Example

Instruction Count = 40 million

Execution Time = 2 seconds

$$MIPS = \frac{40}{2} = 20$$

Processor speed = **20 MIPS**

Advantages

- Easy to calculate
- Compares processors of the same ISA

Limitations

- Cannot compare different architectures accurately.
- Ignores instruction complexity.

3. Amdahl's Law

Definition: Amdahl's Law predicts the maximum improvement in system performance when only part of the system is improved.

Formula

$$Speedup = \frac{1}{(1-f) + \frac{f}{s}}$$

Where

- f = Fraction improved
- s = Speedup of improved portion

Example

Suppose

40% of program is improved.

Improved part becomes 5 times faster.

$$Speedup = \frac{1}{0.6 + 0.08} = 1.47$$

Overall performance increases by **1.47 times**.

Importance

- Identifies bottlenecks
- Helps optimize system performance
- Used in processor design

Real-world ISA Case Study: ARM Cortex vs Intel x86

Introduction

ARM Cortex and Intel x86 are two of the most widely used processor architectures.

ARM Cortex is designed for **low power consumption**, while Intel x86 is designed for **high performance**.

ARM Cortex

ARM Cortex processors are based on **RISC architecture** and are widely used in:

- Smartphones
- Tablets
- Embedded systems
- IoT devices

Examples

- Cortex-A53
- Cortex-A72
- Cortex-M4
- Cortex-R5

Intel x86

Intel x86 processors use **CISC architecture**.

They are widely used in

- Desktop computers
- Laptops
- Servers
- Gaming computers

Examples

- Intel Core i3
- Intel Core i5
- Intel Core i7
- Intel Xeon

Comparison

Feature	ARM Cortex	Intel x86
Architecture	RISC	CISC
Instruction Length	Fixed	Variable
Power Consumption	Low	High
Performance	Good	Very High
Heat Generation	Low	High
Battery Life	Excellent	Moderate
Cost	Low	Higher
Devices	Smartphones, IoT	PCs, Servers
Examples	Cortex-A53, Cortex-M4	Core i5, Core i7

Advantages of ARM Cortex

- Low power consumption
- Long battery life
- Compact processor
- Less heat generation
- High performance per watt

Advantages of Intel x86

- High computing performance
- Supports legacy software
- Better multitasking
- Suitable for gaming
- Large software ecosystem

Applications

ARM Cortex

- Smartphones
- Tablets
- Smart TVs
- Wearable devices
- Embedded systems
- Automotive systems

Intel x86

- Desktop PCs
- Laptops
- Servers
- Workstations
- Gaming computers

ARM Cortex processors are ideal for **mobile and embedded devices** because they consume less power and offer longer battery life. Intel x86 processors are preferred for **desktops, laptops, servers, and gaming systems** where maximum performance is required.

Simple Assembly Programs

1. Addition of Two Numbers (ARM)

```
MOV R0,#20
MOV R1,#30
ADD R2,R0,R1
```

Output: R2 = 50

2. Subtraction of Two Numbers (ARM)

```
MOV R0,#40
MOV R1,#15
SUB R2,R0,R1
```

Output: R2 = 25

3. Multiplication (ARM)

```
MOV R0,#6
MOV R1,#8
MUL R2,R0,R1
```

Output: R2 = 48

4. Addition of Two Numbers (x86)

```
MOV AX,15
MOV BX,25
ADD AX,BX
```

Output: AX = 40

5. Subtraction (x86)

```
MOV AX,30
MOV BX,10
SUB AX,BX
```

Output: AX = 20

6. Find the Largest of Two Numbers (x86)

```
MOV AX,25
MOV BX,40
CMP AX,BX
JG FIRST
MOV CX,BX
JMP END

FIRST:
MOV CX,AX

END:
```

Explanation:

- CMP compares the values in AX and BX.
- JG (Jump if Greater) transfers control to FIRST if AX > BX.
- If AX is not greater, the value in BX is copied to CX.
- The larger number is stored in CX.

These notes follow the exact order of your syllabus and are suitable for university exams, covering definitions, key concepts, tables, examples, advantages, and simple assembly programs.